

0 1 F 3 B 2 7 5 C
A 9 E D

AES-256: THE PURE RAW MATH

From Galois Fields to Ciphertext

Journey into the Cipher

01

01 Why AES? Block Ciphers & SPN

02

02 Parameters: 256-bit Key, 14 Rounds

03

03 The State & Byte Representation

04

04 Round Steps: SubBytes, ShiftRows, MixColumns, AddRoundKey

05

05 S-box Deep Dive ($GF(2^8)$ & Affine Transform)

06

06 Key Expansion for AES-256

07

07 Decryption & Modes

08

08 Security Considerations

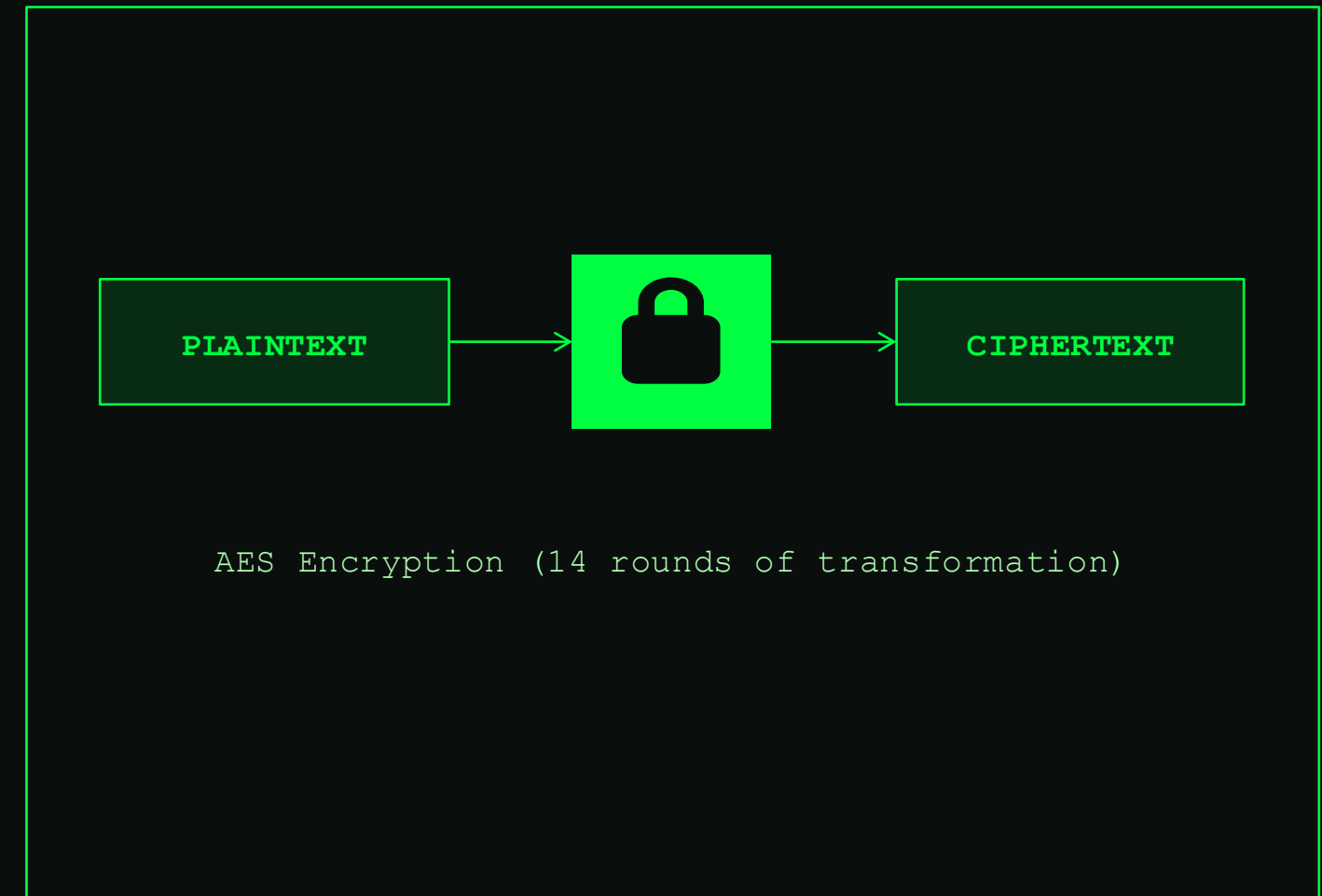
Why AES?

- > Advanced Encryption Standard (2001)
- > Superseded DES (broken by brute force)
- > Symmetric block cipher, 128-bit block
- > Substitution-Permutation Network (SPN)
- > Designed by Joan Daemen & Vincent Rijmen

SPN Architecture:

Alternating layers of **confusion**

(substitution) and **diffusion** (permutation)



AES-256 at a Glance

KEY SIZE

256 bits (32 bytes)

2^{256} possible keys

BLOCK SIZE

128 bits (16 bytes)

4 x 4 byte matrix (the State)

NUMBER OF ROUNDS

14

per NIST FIPS 197

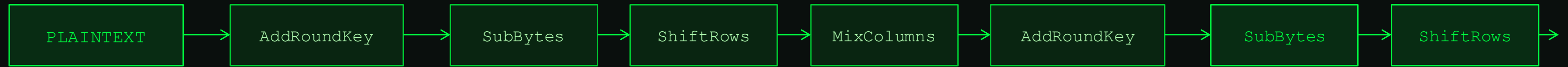
The 4x4 State Matrix:

s[0,0]	s[0,1]	s[0,2]	s[0,3]
s[1,0]	s[1,1]	s[1,2]	s[1,3]
s[2,0]	s[2,1]	s[2,2]	s[2,3]
s[3,0]	s[3,1]	s[3,2]	s[3,3]

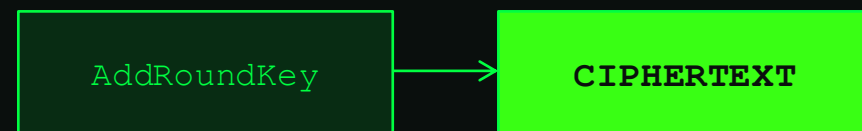
Stored in **column-major order**
Each cell = 1 byte (8 bits)
Total: 16 bytes = 128 bits

All bytes are elements in $GF(2^8)$ with irreducible polynomial:
 $m(x) = x^8 + x^4 + x^3 + x + 1$ (0x11B)

AES Encryption Flow



[repeated 13 times for rounds 1-13]



[Final round – no MixColumns]

Round Structure Detail:

Initial Round (Round 0): AddRoundKey only – XOR plaintext with round key 0

Standard Rounds (1-13): 1. SubBytes -> 2. ShiftRows -> 3. MixColumns -> 4. AddRoundKey

Final Round (14): SubBytes -> ShiftRows -> AddRoundKey (no MixColumns)

Omitting MixColumns in final round ensures cipher is invertible for decryption.

The 4x4 Byte State

State $s[r, c]$ – column-major:

$s[0,0]$	$s[0,1]$	$s[0,2]$	$s[0,3]$
$s[1,0]$	$s[1,1]$	$s[1,2]$	$s[1,3]$
$s[2,0]$	$s[2,1]$	$s[2,2]$	$s[2,3]$
$s[3,0]$	$s[3,1]$	$s[3,2]$	$s[3,3]$

Input byte order:

$in[0] \rightarrow s[0,0]$ $in[4] \rightarrow s[0,1]$
 $in[1] \rightarrow s[1,0]$ $in[5] \rightarrow s[1,1]$
 $in[2] \rightarrow s[2,0]$...
 $in[3] \rightarrow s[3,0]$ $in[15] \rightarrow s[3,3]$

$GF(2^8)$ – The Galois Field

Each byte is a polynomial in $GF(2^8)$

Example: $0x57$
 $= 0x57$
 $= x^6 + x^4 + x^2 + x + 1$

Irreducible polynomial:

$m(x) = x^8 + x^4 + x^3 + x + 1$
 $= 0x11B \text{ (hex)}$

Field Operations:

Addition : XOR (no carry)
Multiplication: polynomial mult + mod $m(x)$
Inversion : extended Euclidean algorithm

SubBytes – The Heart of Confusion

```
state[i][j] = S[state[i][j]]    (16x16 lookup table)
```

S-box Construction:

- 1. Multiplicative inverse in GF(2⁸)
 $inv(x) = x^{-1} \bmod m(x)$ [0x00 -> 0x00]
- 2. Affine transform over GF(2):
 $b = A * inv + c$ where $c = 0x63$

Matrix A (8x8):

1	0	0	0	1	1	1	1
1	1	0	0	0	1	1	1
1	1	1	0	0	0	1	1
1	1	1	1	0	0	0	1
1	1	1	1	1	0	0	0
0	1	1	1	1	1	0	0
0	0	1	1	1	1	1	0
0	0	0	1	1	1	1	1

Example: 0x53 -> 0xED

0x53 in GF(2⁸) -> inverse = 0xCA

Affine(0xCA) with c=0x63 -> result = 0xED

The AES S-Box (16 x 16)

63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16

ShiftRows – Diffusion Across Columns

Before:

a00	a01	a02	a03
a10	a11	a12	a13
a20	a21	a22	a23
a30	a31	a32	a33

->

After:

a00	a01	a02	a03
a11	a12	a13	a10
a22	a23	a20	a21
a33	a30	a31	a32

Row 0: shift left by 0 (unchanged)

Row 1: shift left by 1 byte

Row 2: shift left by 2 bytes

Row 3: shift left by 3 bytes

Visual: Row Shifts

R0: [a00][a01][a02][a03] (no shift)
R1: [a10][a11][a12][a13] -> [a11][a12][a13][a10]
R2: [a20][a21][a22][a23] -> [a22][a23][a20][a21]
R3: [a30][a31][a32][a33] -> [a33][a30][a31][a32]

Bytes from one column now spread
across multiple columns

MixColumns – Matrix Mult in GF(2⁸)

$s' = M * s$ (per column)

Fixed matrix M:

02	03	01	01
01	02	03	01
01	01	02	03
03	01	01	02

xtime – Multiply by 02:

$\text{xtime}(b) = (b \ll 1)$

if (MSB of $b == 1$): XOR with 0x1B

Example: $\text{xtime}(0x57)$

$0x57 \ll 1 = 0xAE$

MSB(0x57) = 1 $\rightarrow$ XOR 0x1B

Result: $0xAE \text{ xor } 0x1B = 0xB5$

Multiplication Rules in GF(2⁸):

* 01 = identity (no change)

* 02 = $\text{xtime}(b)$

* 03 = $\text{xtime}(b) \text{ XOR } b$

* 09 = $\text{xtime}(\text{xtime}(\text{xtime}(b))) \text{ XOR } b$

All products reduced mod $m(x) = 0x11B$

Full Column Computation:

$s'[0] = 02*s[0] \text{ xor } 03*s[1]$
 $\text{ xor } 01*s[2] \text{ xor } 01*s[3]$

$s'[1] = 01*s[0] \text{ xor } 02*s[1]$
 $\text{ xor } 03*s[2] \text{ xor } 01*s[3]$

$s'[2] = 01*s[0] \text{ xor } 01*s[1]$
 $\text{ xor } 02*s[2] \text{ xor } 03*s[3]$

$s'[3] = 03*s[0] \text{ xor } 01*s[1]$
 $\text{ xor } 01*s[2] \text{ xor } 02*s[3]$

AddRoundKey – Simple XOR

```
state[r,c] = state[r,c] XOR roundKey[r,c]
```

State				Round Key				Result			
32	88	31	E0	2B	28	AB	09	19	A0	9A	E9
43	5A	31	37	7E	AE	F7	CF	3D	F4	C6	F8
F6	30	98	07	15	D2	15	4F	E3	E2	8D	48
A8	8D	A2	34	16	A6	88	3C	BE	2B	2A	08

Key Properties:

+ AddRoundKey is its own inverse: (a XOR k) XOR k = a

+ Each round uses a unique subkey from the key expansion schedule

+ This is where the secret key material enters the cipher – without it, all other steps are publicly known and reversible

+ Round keys come from key expansion (next slide)

Key Expansion: 256-bit -> 60 words

32-byte key -> W[0] ... W[7] (8 initial words)

```
Algorithm (i = 8 to 59):  
  
  if i mod 8 == 0:  
    temp = SubWord(RotWord(W[i-1]))  
           xor Rcon[i/8]  
  else if i mod 8 == 4:  <-- AES-256 only  
    temp = SubWord(W[i-1])  
  else:  
    temp = W[i-1]  
  
  W[i] = W[i-8] xor temp
```

Rcon values:

i	1	2	3	4	5	6	7	8	...
Rcon	01	02	04	08	10	20	40	80	1B

Key difference for AES-256: Extra SubWord at $i \bmod 8 == 4$
adds nonlinearity to strengthen against related-key attacks.

Key Schedule Flow

Input:

32-byte cipher key K[0..31]

Step 1 – Split:

W[0..7] = K[0..31] as 8 words

Step 2 – Expand:

For i = 8 to 59:

 compute temp per rules above

 W[i] = W[i-8] xor temp

Step 3 – Form keys:

RoundKey[r] = W[4r..4r+3]

for r = 0, 1, ..., 14

Output:

15 round keys (60 words total)

AES-256 in Action

PT = 32 43 F6 A8 88 5A 30 8D 31 31 98 A2 E0 37 07 34 (16 bytes) Key = 256-bit sample key

1. Initial State

32	88	31	E0
43	5A	31	37
F6	30	98	07
A8	8D	A2	34

2. AddRoundKey(0)

19	A0	9A	E9
3D	F4	C6	F8
E3	E2	8D	48
BE	2B	2A	08

3. SubBytes

D4	E0	B8	1E
27	BF	B4	41
11	98	5D	52
AE	F1	E5	30

4. ShiftRows

D4	E0	B8	1E
BF	B4	41	27
5D	52	11	98
30	AE	F1	E5

5. MixColumns

04	E0	48	28
66	CB	F8	06
81	19	D3	26
E5	9A	7A	4C

6. AddRoundKey(1)

A4	68	6B	02
9C	9F	5B	6A
7F	35	EA	50
F2	2B	43	49

Highlighted bytes changed position in ShiftRows

Normal bytes show progression through steps

Each step completely transforms the state

Round 1 Complete – What happened:

SubBytes: Every byte nonlinearly transformed via S-box (confusion)

ShiftRows: Bytes redistributed across columns (diffusion layer 1)

MixColumns: Each column mixed via matrix multiplication (diffusion layer 2)

AddRoundKey: Key material XORed into state (key introduction)

Decryption: Inverse Operations

Inverse Operations Table:

Encrypt	Decrypt	Description
SubBytes	InvSubBytes	Inverse S-box lookup
ShiftRows	InvShiftRows	Shift right (0,1,2,3)
MixColumns	InvMixColumns	Mult by inverse matrix

Inverse MixColumns Matrix:

0E	0B	0D	09
09	0E	0B	0D
0D	09	0E	0B
0B	0D	09	0E

Decryption Flow:

CT -> InvAddRoundKey -> InvMixColumns -> InvShiftRows ->
InvSubBytes (repeat) -> PT

Note: order reversed because operations don't commute

Why This Works:

InvSubBytes:

$S^{-1}[S[x]] = x$ for all x
Inverse S-box precomputed similarly

InvShiftRows:

Shift right by (0,1,2,3)
Reverses the left shifts

InvMixColumns:

Uses inverse matrix shown left
 $M^{-1} * M = I$ in $GF(2^8)$

Equivalent Inverse Cipher:

Can reorder InvShiftRows and
InvSubBytes (they commute)
to match encryption structure

Beyond ECB – Block Cipher Modes



ECB

INSECURE – Do Not Use

Electronic Codebook
Each block encrypted
independently

Problem:

Identical PT = Identical CT
Patterns leak through



CBC

Cipher Block Chaining

Each block XORed with
previous ciphertext
before encryption

Requires:

Random IV (16 bytes)
PKCS#7 padding



GCM

Galois/Counter Mode

Authenticated Encryption
(AEAD)

Provides:

Confidentiality + Integrity
Authentication tag (128-bit)
Most widely used today

Practical Guidance:

- > Always use an **unpredictable IV/nonce** – never reuse with the same key in CBC
- > Use **PKCS#7 padding** for CBC: pad with value = number of padding bytes
- > Prefer **GCM** for new systems – it provides both encryption and integrity verification
- > Never use ECB mode for data that may contain patterns or repetition

Why AES-256 Remains Strong

1 Brute force: 2^{256} operations

Physically infeasible – more combinations than atoms in the observable universe

2 Related-key attacks: theoretical only

Complexity above 2^{100} – no practical attack demonstrated to date

3 Side-channel resistance

Constant-time implementations prevent timing and power analysis attacks

Security Summary

No practical attack on full AES-256

Best known attack:

Biclique attack on full rounds

with complexity $\sim 2^{254.4}$

Only 1.6x faster than

brute force – negligible gain

Math that protects the digital world.

Now go build your own.